

Friedrich-Alexander University Erlangen-Nuremberg

# **Rule-based learning for generating semantic descriptions of patient subgroups using RuleKit**

Biomedical Network Science Project (Summer 2025)

**Prepared by:**

Ibne Sayad

Matriculation number: 23365652

Master of Science in Artificial Intelligence

Email: [ibne.sayad@fau.de](mailto:ibne.sayad@fau.de)

**Supervised by:**

Fabian Woller, M.Sc.

Department of Artificial Intelligence in Biomedical Engineering (AIBE)

Biomedical Network Science Lab

Email: [fabian.woller@fau.de](mailto:fabian.woller@fau.de)

## Table of contents

|                                    |            |
|------------------------------------|------------|
| <b>Abstract</b>                    | <b>iii</b> |
| <b>1. Introduction</b>             | <b>1</b>   |
| <b>2. Methods</b>                  | <b>2-4</b> |
| i. RuleKit Functionality           | 2          |
| ii. Workflow                       | 2          |
| iii. Packaging                     | 3          |
| <b>3. Benchmarking and Testing</b> | <b>5-9</b> |
| i. Testing Setup                   | 5          |
| ii. Unit Test Results              | 5          |
| iii. Benchmarking Setup            | 7          |
| iv. Benchmarking Result            | 7          |
| <b>4. Conclusion</b>               | <b>10</b>  |
| <b>5. Reference</b>                | <b>11</b>  |

## Abstract

Clinical and multi-omics datasets are inherently complex due to their heterogeneous nature, often containing a combination of numerical and categorical variables, as well as a significant proportion of missing values. This complexity poses considerable challenges in analysis and interpretability. Clustering methods are frequently employed as an initial step to uncover latent structures within such data. However, interpreting the resulting clusters and identifying meaningful rules that define them remains a difficult task. This project aims to address this gap by developing a rule-based learning approach to extract interpretable and semantically rich descriptions for each cluster. We are working with a dataset where clustering has already been performed, and we are applying either manual or automated methods. Our goal is to understand the cluster. Essentially, we attempted to explain them using a set of straightforward, easy-to-understand rules. The hope was that these rules would tell us something useful about what makes one group different from another. Which a human can understand and work further accordingly. To accomplish that, we used a tool called RuleKit. It's a supervised rule-based learning system that works well with different types of data, including both categories and numbers, even as ranges. Importantly, we didn't impute missing values. Rather, we kept them as they were, since they might carry some meaning. To operate this approach, we developed a dedicated Python package, RuleFetcher. This is publicly available at <https://github.com/Ibne-Sayad/Rule-Based-Learning-RuleKit>. RuleFetcher integrates with RuleKit to automatically generate interpretable rule sets for clustered data. The functionality of the package was validated through unit tests, and its scalability was assessed using runtime benchmarking on both real and synthetic datasets. The overarching objective of this work is to enhance the interpretability of clustering results, enabling researchers to understand better, validate, and critically assess the groupings identified. By uncovering explicit, rule-based representations of each cluster, this approach provides transparent and explainable insights that support expert-driven analysis. Unlike black-box models, the proposed method offers interpretable logic that facilitates informed decision-making and fosters trust in downstream analyses.

**Key Words:** Rule-based learning, RuleFetcher, RuleKit, Cluster analysis, Semantic subgroup discovery, Explainable AI

## 1. Introduction

Biomedical research has entered a modern era in which researchers compile datasets from a wide range of biological, clinical, and demographic sources. Such datasets are rich in potential but come with their own complexities. Typically, they combine categorical variables, such as gender, diagnosis status, and tumor types, with continuous values, like age, blood pressure, or gene expression levels. Analyzing this kind of mixed-type data introduces various challenges. Many conventional machine learning techniques struggle to perform effectively on such hybrid data types [6], [10]. One added obstacle is the frequent presence of missing data, often resulting from human or technical errors during data collection. However, in clinical contexts, missingness may itself carry hidden significance [7]. For this reason, our approach avoids indiscriminate imputation or deletion of missing values and instead handles them explicitly.

This project introduces RuleFetcher, a modular Python package developed to meet the needs of interpretable learning in biomedical data. RuleFetcher builds upon RuleKit [3], [4], a rule-based machine learning engine, and adds a user-friendly, end-to-end workflow that includes cluster annotation, rule extraction, and semantic interpretation. The package is designed to provide transparent, explainable models and is especially suited for datasets where interpretability and trust are critical, such as in clinical decision support [1], [2], [5]. Throughout this report, we demonstrate how RuleFetcher is used to train rule-based classifiers and map rules to clusters, allowing meaningful descriptions of patient subgroups.

Rule-based learning, a form of symbolic, interpretable machine learning, produces simple IF–THEN rules from data [3], [6], [9]. Unlike black-box models, it allows domain experts to follow the logic behind predictions, which is crucial in high-stakes environments like medicine [5], [7]. Instead of merely outputting a prediction, the system translates recurring patterns in the data into human-readable conditions that explain why certain decisions are made. For example, a rule might state: “IF the patient is older than 60 AND a certain biomarker is present, THEN the patient belongs to Cluster A.” This direct reasoning process makes rule-based models valuable not only for prediction but also for post hoc analysis, validation, and insight generation [1], [10].

In our project, rule-based learning serves as the backbone for interpreting and validating clusters in a biomedical dataset. While clustering algorithms can group patients based on statistical similarity, their utility remains limited if the meaning of those clusters is unclear. Clinical or biological actions require understandable and justifiable subgroup definitions [8]. By applying RuleKit through our RuleFetcher package, we can generate rule-based explanations for each cluster, thereby supporting transparency, interpretability, and trust. Importantly, RuleKit respects missing values by treating them as unknowns rather than forcing imputations.[3], [4].

This methodology not only enables validation of given input clusters but also uncovers clinically meaningful patterns that might have remained hidden. In a domain where interpretability is often

sacrificed for performance, RuleFetcher offers a practical balance by preserving explainability without compromising analytical power.

## **2. Methods**

### **i. RuleKit Functionality**

In our pipeline, RuleKit served as the engine for rule induction, enabling interpretable pattern extraction from the clustered biomedical dataset. RuleKit is a Java-based tool specifically designed to learn IF–THEN rules from structured tabular data [3], [4]. It supports a range of induction and pruning strategies, allowing the user to adapt it to various learning scenarios. One of the key reasons for selecting RuleKit was its ability to operate effectively on mixed-type datasets, which are common in biomedical applications. More importantly, it handles missing values (NAs) in a principled way. Rather than relying on deletion or imputation, RuleKit uses the standard ARFF file format, where missing entries are marked with a “?” symbol [3], [4]. During rule induction, these unknowns are excluded from the evaluation of the conditions they affect. Concretely, this means that if a condition in a rule involves an attribute that is missing in each instance, that particular condition is ignored for that instance. The instance itself does not contribute toward supporting or contradicting the rule. This design avoids the introduction of bias through imputation and preserves the integrity of the learning process, especially in clinical datasets where missingness can carry implicit meaning [3].

In this project, we experimented with several rule quality functions supported by RuleKit, including C2, Correlation, and RSS [1], [3], [4]. C2 is based on conditional entropy and prioritizes rules that reduce uncertainty about the target variable. Correlation favors combinations of attributes that frequently co-occur within the same class, thereby surfacing feature patterns that are tightly linked to a specific group. RSS (Relative Support and Significance) highlights rules that either apply to a large portion of the data (support) or distinguish classes with high precision (significance). While these measures were used in parallel across different runs, no formal benchmarking was conducted to compare their performance quantitatively. Instead, they were applied to explore qualitatively how each measure influenced the types of rules generated and how well those rules captured the structural characteristics of the clusters.

### **ii. Workflow**

The RuleFetcher pipeline is designed as a modular and repeatable process that transforms a raw dataset with cluster annotations into semantically interpretable rules. The process begins with a clustered CSV file, which contains both feature columns and a clustering label. If the dataset does not already include cluster assignments, the pipeline optionally supports automated clustering

(e.g., via K-modes or other unsupervised techniques), allowing flexibility depending on the input scenario.

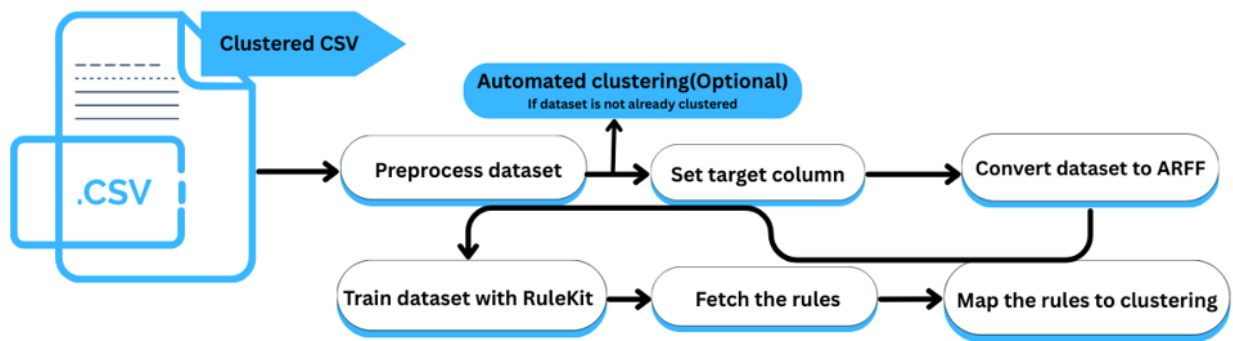
The dataset then undergoes preprocessing, which includes cleaning column names, handling missing values, encoding categorical features, and ensuring compatibility with RuleKit's format. Preprocessing also allows the user to define the target column, i.e., the variable representing the cluster label that will be used for rule learning.

Once the target is set, the dataset is converted into ARFF format, which is the standard input for RuleKit. This format retains type information for both categorical and numerical features and represents missing values using the “?” symbol, allowing RuleKit to process them without requiring imputation [3], [4]. The clean ARFF file is then passed to the RuleKit engine for training.

During training, RuleKit applies rule induction algorithms such as C2, Correlation, or RSS, each tailored to capture different types of regularities or patterns in the data [1], [3], [4]. After training, the rules are extracted along with key metrics like support, precision, and coverage, providing quantitative insight into each rule's reliability.

Finally, the extracted rules are mapped back to the original cluster labels, enabling semantic interpretation of the cluster structures. This mapping step bridges the gap between statistical clustering and explainable machine learning, helping users understand what defines each subgroup in practical terms. This interpretability is particularly valuable in biomedical contexts where transparency and reproducibility are essential [1], [2], [5].

The complete updated workflow is illustrated in Figure 01, showcasing the modular transformation from clustered data to interpretable subgroup rules.



**Figure 01:** The updated workflow of the rule-based interpretation pipeline starts with a clustered CSV file. After preprocessing and optionally applying automated clustering, the target column is set and the data is converted to ARFF format. RuleKit is then used to train on the data, extract rules, and map them back to clusters for semantic interpretation.

### iii. Packaging

As part of this project, the entire rule-based learning pipeline has been developed and published as a reusable Python package named RuleFetcher. The package provides a streamlined, end-to-end workflow for interpretable subgroup analysis by integrating data preprocessing, clustering, ARFF conversion, rule induction using RuleKit, rule-to-cluster mapping, and performance evaluation. It is publicly available on GitHub and can be directly installed using pip via the command-

**`pip install git+https://github.com/Ibne-Sayad/Rule-Based-Learning-RuleKit.git`**

This makes the workflow accessible to other researchers and supports reproducibility and collaborative development.

The pipeline accepts as input a CSV file containing structured, patient-level data composed of both categorical and numerical attributes. This file must include either pre-assigned cluster labels or be suitable for automatic clustering within the workflow. To ensure smooth execution, the user can configure the preprocessing step to rename columns to standard identifiers and remove any unnecessary fields. After preprocessing, the cleaned data is converted to the ARFF format, which RuleKit requires for compatibility. This conversion preserves the mixed-type structure of the data and marks missing values with the standard "?" symbol, thereby allowing RuleKit to handle them properly during rule induction [3], [4].

Once the ARFF file is prepared, RuleFetcher uses it to train rule-based models using a selected induction algorithm, such as C2, Correlation, or RSS. The choice of algorithm is passed as a parameter, allowing users to experiment with different rule qualities. The rules are extracted from the trained models and saved together with metrics such as rule support, confidence, precision, and rule length. These rules are then mapped back to their respective clusters, enabling semantic interpretation of each group based on human-readable patterns. The pipeline also generates tabular summaries that describe the rule statistics and highlight the coverage of each rule set with respect to the clusters.

Internally, the package is modular and organized into three main components. The DataHandler class is responsible for loading the dataset, renaming and cleaning columns, preserving missingness, and encoding categorical variables. The Clusterer module supports both clustering with KModes and semantic labeling using decision trees, although this is an optional step if automated clustering is required. The RuleKitWrapper class handles conversion to ARFF format, interaction with RuleKit, model training, and rule extraction. Together, these modules encapsulate the full learning cycle and can be reused or extended for other datasets and learning tasks.

By publishing the package and enabling parameterized execution, this project contributes to the growing need for transparent and interpretable machine learning in biomedical research. RuleFetcher not only supports the replication of the results presented here but also allows researchers to apply the same methodology to new domains, fostering greater confidence and understanding in rule-based subgroup discovery [1], [3], [4].

### **3. Benchmarking and Testing**

#### **i. Testing Setup**

The dataset used in this study originates from the publicly available TCGA Pan-Cancer clinical dataset, which contains multi-omics data for thousands of patients across multiple tumor types [11]. The dataset size is approximately 12,500 samples and 34 variables. It includes a combination of categorical features, such as cancer type abbreviation, gender, race, and treatment outcome, alongside numerical features like age at initial pathologic diagnosis, overall survival time, and disease-specific survival time. A considerable portion of the dataset contains missing values, which were intentionally retained to simulate real-world clinical data environments where missingness itself may carry implicit meaning.

To prepare the dataset for rule-based learning, several structured steps were applied in the pipeline. Column names were standardized using a renaming schema to ensure consistency with the internal processing modules, and irrelevant or highly missing fields were excluded from analysis. The cleaned dataset was then subjected to clustering, either through manual labels or using the KModes algorithm, as implemented in the pipeline. Following clustering, the cluster labels were added to the dataset to allow later rule-based interpretation of each subgroup. The final cleaned and clustered dataset was converted into the ARFF format required by RuleKit. This conversion preserved both categorical and numerical data types and encoded missing entries using the standard question mark symbol (?), enabling RuleKit to process them correctly during rule induction.

The testing setup was orchestrated through a modular Python script that executes the full pipeline using RuleFetcher. The script loads the CSV file, applies column cleaning and renaming logic, performs clustering, and exports the clustered dataset to ARFF format. RuleKitWrapper is then initialized with the clustered file, which triggers ARFF writing, rule induction, and summary metric extraction. This controlled and repeatable setup ensures consistency across experiments and enables structured rule generation from mixed-type biomedical data.

#### **ii. Unit Test Results**

To validate the functionality of the pipeline and ensure correct rule extraction, a set of unit tests was conducted using a manually created suite of six clustered datasets. Each dataset was designed to reflect a specific combination of variables such as age, gender, cancer type, and vital status. After preprocessing, each dataset was passed through the RuleKitWrapper module, which executed three different rule induction algorithms- C2, Correlation, and RSS. The rules generated by each method were successfully mapped back to the original cluster IDs, as shown in the output logs. For clarity and verification, the results were also compiled into a tabular summary (Table 01) where each row corresponds to a cluster, listing the defining conditions, expected ground truth rule, and the rules fetched from each algorithm. The table includes six logical cluster pairs, each



with cluster IDs 0 and 1, representing the two sides of a binary condition, such as age above or below a threshold, or the presence or absence of a specific cancer type. The outcome shows that for all clusters, the rule-fetching mechanism correctly identified the underlying logic, aligned with the expected ground truth, and returned interpretable rules in each case.

| Cluster     | Age | Gender | CancerType | VitalStatus | Ground truth rule |    |     | #Fetched rules |    |     |
|-------------|-----|--------|------------|-------------|-------------------|----|-----|----------------|----|-----|
|             |     |        |            |             | C2                | CR | RSS | C2             | CR | RSS |
| Cluster 1-0 | >40 | -      | -          | -           | ✓                 | ✓  | ✓   | 01             | 01 | 01  |
| Cluster 1-1 | <40 | -      | -          | -           | ✓                 | ✓  | ✓   | 01             | 01 | 01  |
| Cluster 2-0 | -   | -      | ! BLCA     | -           | ✓                 | ✓  | ✓   | 15             | 14 | 16  |
| Cluster 2-1 | -   | -      | BLCA       | -           | ✓                 | ✓  | ✓   | 01             | 01 | 01  |
| Cluster 3-0 | >50 | Male   | -          | Alive       | ✓                 | ✓  | ✓   | 02             | 02 | 02  |
| Cluster 3-1 | <50 | Female | -          | Dead        | ✓                 | ✓  | ✓   | 08             | 06 | 03  |
| Cluster 4-0 | <40 | Female | -          | -           | ✓                 | ✓  | ✓   | 02             | 02 | 02  |
| Cluster 4-1 | <40 | Male   | -          | -           | ✓                 | ✓  | ✓   | 04             | 02 | 02  |
| Cluster 5-0 | -   | Female | -          | -           | ✓                 | ✓  | ✓   | 01             | 01 | 01  |
| Cluster 5-1 | -   | Male   | -          | -           | ✓                 | ✓  | ✓   | 01             | 01 | 01  |
| Cluster 6-0 | -   | -      | -          | Alive       | ✓                 | ✓  | ✓   | 01             | 01 | 01  |
| Cluster 6-1 | -   | -      | -          | Dead        | ✓                 | ✓  | ✓   | 01             | 01 | 01  |

**Table 01:** It summarizes the unit test setup, where each cluster represents a logical subgroup defined by specific conditions such as age, gender, cancer type, or vital status. For each cluster pair, the table indicates whether the expected ground truth rule was correctly captured, and the number of rules retrieved from each of the three algorithms (C2, Correlation, and RSS). The results confirm that the rule-fetching logic aligns with predefined subgroup definitions across all test cases.

### iii. Benchmarking Setup

To evaluate the scalability of the rule-based learning workflow, a comprehensive benchmarking setup was prepared using synthetic datasets derived from the original TCGA Pan-Cancer dataset. The original dataset served as the statistical template for generating simulated features and samples. Specifically, five levels of feature dimensionality (100, 200, 300, 400, and 500 variables) and four sample sizes (5,000, 10,000, 15,000, and 20,000 samples) were used, resulting in 20 unique dataset configurations. For each configuration, five independent datasets were generated, totaling 100 datasets overall. Categorical values in the synthetic data were sampled randomly from the existing categories in the original dataset, while numerical values were drawn from a normal distribution that matched the mean and standard deviation of their respective reference columns. A cluster label was then randomly assigned to each row, simulating a classification task without enforcing any rule-based structure.

These datasets were used to benchmark the full pipeline implemented in the RuleFetcher package, which internally interfaces with RuleKit for rule induction. The benchmarking procedure measured the complete runtime of each dataset through the core stages of the pipeline, including data preprocessing, ARFF conversion, model training, and rule extraction. Each dataset was processed five times to account for technical artifacts such as fluctuations in system load and I/O performance. The resulting runtime data was stored in CSV format, capturing both individual run durations and summary statistics across repetitions. This setup allowed us to systematically explore how execution time scales with increasing dataset size and complexity while maintaining consistency across test conditions.

### iv. Benchmarking Result

The runtime results in Table 02 highlight how the RuleFetcher pipeline responds to increasing dataset sizes across two dimensions: number of variables and number of samples. As shown, the runtime grows progressively as either dimension increases, but the rate of growth is not uniform. For a fixed sample size, increasing the number of variables leads to a much steeper rise in execution time compared to increasing the number of samples at a fixed variable count. For instance, moving from 100 to 500 variables at 5,000 samples increases the average runtime from 0.35 to 5.38 minutes, while moving from 5,000 to 20,000 samples at 100 variables only increases it from 0.35 to 1.91 minutes. This pattern is consistent across the table and reveals that feature dimensionality is the dominant factor influencing runtime.

This trend reflects the nature of rule-based learning, where more features directly expand the search space for rule induction, leading to higher computational overhead. The additional time cost from increasing sample size is comparatively moderate because most rule learners, including RuleKit, process samples linearly but experience exponential growth with feature interactions [3], [4]. These results suggest that while the pipeline remains efficient for large sample volumes, high-

dimensional feature spaces require significantly more time and should be managed carefully, especially in real-world biomedical applications where hundreds of variables are common.

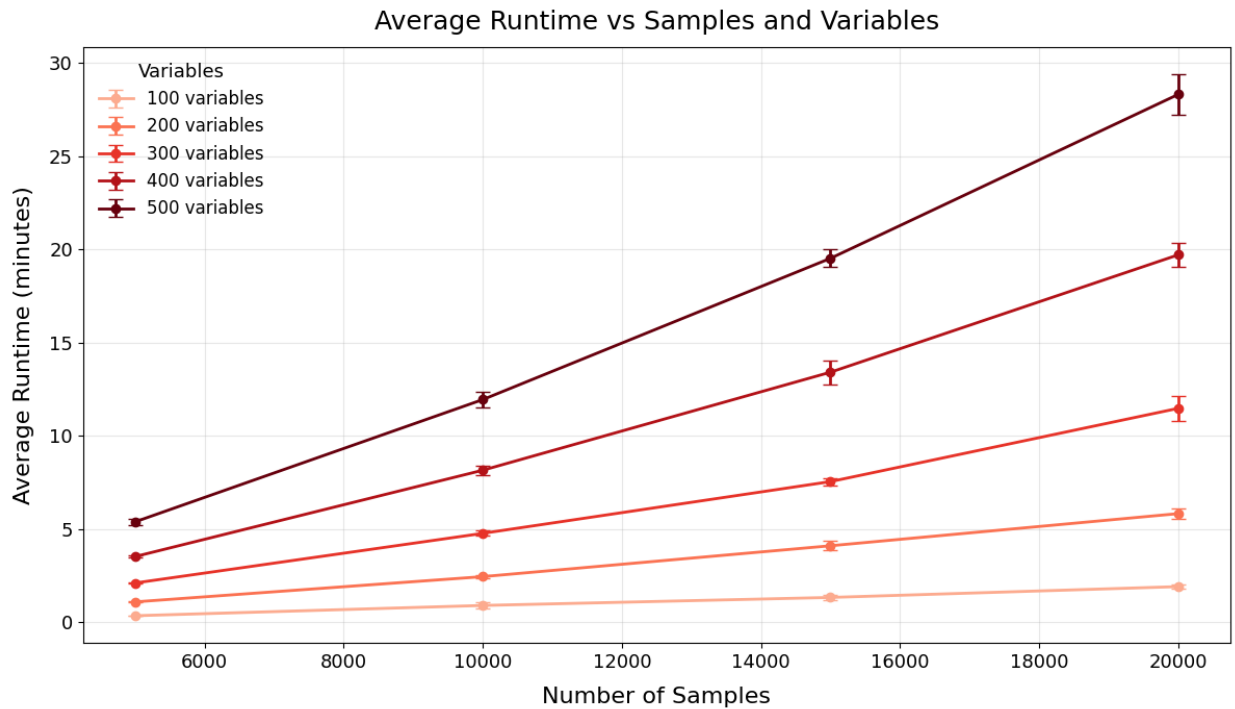
| Variables | Samples | Min. Runtime | Max. Runtime | Avg. Runtime |
|-----------|---------|--------------|--------------|--------------|
| 100       | 5000    | 00.33 min    | 00.37 min    | 00.35 min    |
| 100       | 10000   | 00.77 min    | 01.13 min    | 00.90 min    |
| 100       | 15000   | 01.21 min    | 01.48 min    | 01.33 min    |
| 100       | 20000   | 01.81 min    | 01.99 min    | 01.91 min    |
| 200       | 5000    | 01.02 min    | 01.13 min    | 01.09 min    |
| 200       | 10000   | 02.37 min    | 02.52 min    | 02.45 min    |
| 200       | 15000   | 03.75 min    | 04.44 min    | 04.10 min    |
| 200       | 20000   | 05.46 min    | 06.09 min    | 05.83 min    |
| 300       | 5000    | 02.04 min    | 02.15 min    | 02.10 min    |
| 300       | 10000   | 04.55 min    | 04.89 min    | 04.77 min    |
| 300       | 15000   | 07.37 min    | 07.79 min    | 07.54 min    |
| 300       | 20000   | 10.68 min    | 12.54 min    | 11.48 min    |
| 400       | 5000    | 03.45 min    | 03.57 min    | 03.52 min    |
| 400       | 10000   | 07.87 min    | 08.43 min    | 08.15 min    |
| 400       | 15000   | 12.54 min    | 14.18 min    | 13.41 min    |
| 400       | 20000   | 19.01 min    | 20.57 min    | 19.71 min    |
| 500       | 5000    | 05.22 min    | 05.55 min    | 05.38 min    |
| 500       | 10000   | 11.32 min    | 12.47 min    | 11.95 min    |
| 500       | 15000   | 18.69 min    | 19.80 min    | 19.52 min    |
| 500       | 20000   | 27.20 min    | 29.82 min    | 28.32 min    |

**Table 02:** It shows the minimum, maximum, and average runtime (in minutes) of the pipeline across datasets with varying numbers of samples and variables. As expected, runtime increases steadily with both feature dimensionality and dataset size, with the number of variables having a more pronounced impact on total execution time.

Figure 02 presents a visual summary of how average runtime scales with increasing numbers of samples and variables in the RuleFetcher pipeline. Each curve corresponds to a fixed number of variables, while the x-axis represents growing sample sizes. The plot clearly demonstrates that increasing the number of features has a much greater impact on runtime than increasing the number of samples. For all variable settings, the runtime increases nearly linearly with the number of samples, but the slope becomes significantly steeper as the variable count increases. At 100 variables, processing time grows modestly across sample sizes, while at 500 variables, the runtime escalates sharply, from around 5 minutes at 5,000 samples to over 28 minutes at 20,000 samples.

This divergence highlights the computational cost associated with high-dimensional data. In rule-based systems like RuleKit, a larger number of variables expands the search space for candidate

rules, which directly affects training time. Conversely, the number of samples has a more limited effect, contributing primarily to the statistical evaluation of candidate rules rather than the complexity of rule generation itself. The narrow error bars across all data points further indicate that these trends are consistent and reproducible, with minimal variability due to runtime noise or system fluctuations. Taken together, the results confirm that feature dimensionality is the dominant factor influencing runtime performance in rule-based learning workflows [3], [4].



**Figure 02:** The figure visualizes the average runtime of the pipeline across datasets with increasing numbers of samples and variables. Each curve represents a different feature dimensionality, and the error bars show variation across five runs. The results clearly indicate that runtime grows more steeply with higher variable counts than with larger sample sizes, emphasizing the computational cost of handling high-dimensional data.

## 4. Conclusion

The results of this project demonstrate the practical value of rule-based learning for post-hoc interpretation of clusters in large, mixed-type biomedical datasets. By preserving missing values, supporting both categorical and numerical attributes, and generating transparent IF–THEN rules mapped back to clusters, the approach enables interpretable insights that go beyond traditional black-box validation techniques. This enhances a researcher's ability to understand the structure and semantics of discovered subgroups, uncovering clinically or biologically meaningful patterns.

The pipeline developed in this project has been implemented as an open-source Python package, RuleFetcher, which automates each stage from data preprocessing to rule extraction and mapping. Runtime benchmarking across 100 synthetic datasets confirmed that while the system scales well with larger sample sizes, feature dimensionality poses a more significant computational challenge. These findings point to important trade-offs in applying rule-based learning at scale and highlight directions for future optimization.

Further extensions could include exploring additional rule induction heuristics, integrating visual explanations of rule coverage and overlap, and developing an interactive interface to support real-time model refinement. Beyond its initial focus on multi-omics data, the framework is adaptable to other domains, such as clinical trial analysis, phenotypic subgrouping, and socio-demographic studies. Any setting where structured clusters exist and transparency is essential.

## Declaration

The author has utilized Grammarly and the OpenAI GPT model to enhance the clarity, grammar, and overall writing quality of this paper. The tools were employed for linguistic refinement and not for generating the core scientific content.

## 5. References

- [1] A. Gruca, J. Henzel, I. Kostorz, T. Sterclik, Ł. Wróbel, and M. Sikora, "MAINE: A web tool for multi-omics feature selection and rule-based data exploration," *\*Bioinformatics\**, vol. 38, no. 2, pp. 1773–1775, 2021, doi: 10.1093/bioinformatics/btab687.
- [2] G. T. Berge, O. C. Granmo, T. O. Tveit, A. L. Ruthjersen, and J. Sharma, "Combining unsupervised, supervised and rule-based learning: The case of detecting patient allergies in electronic health records," *\*BMC Med. Inform. Decis. Mak.\**, vol. 23, no. 188, 2023, doi: 10.1186/s12911-023-02271-8.
- [3] M. Sikora, A. Gruca, and Ł. Wróbel, "RuleKit: A comprehensive suite for rule-based learning," *\*SoftwareX\**, vol. 20, p. 101259, 2022, doi: 10.1016/j.softx.2022.101259.
- [4] A. Gruca, M. Sikora, and Ł. Wróbel, "RuleKit 2: Faster and simpler rule learning," *\*J. Comput. Sci.\**, vol. 67, p. 101703, 2022, doi: 10.1016/j.jocs.2022.101703.
- [5] F. Degenhardt and H. Seibold, "Interpretable subgroup learning-based modeling framework," *\*Stat. Methods Med. Res.\**, vol. 30, no. 9, pp. 1995–2010, 2021, doi: 10.1177/09622802211015304.
- [6] S. Patel and S. Joshi, "Graph- and rule-based learning algorithms: A comprehensive review of their applications for cancer type classification and prognosis using genomic data," *\*Front. Genet.\**, vol. 12, p. 689345, 2021, doi: 10.3389/fgene.2021.689345.
- [7] I. Lipkovich, A. Dmitrienko, and R. D'Agostino, "Tutorial in biostatistics: Data-driven subgroup identification and analysis in clinical trials," *\*Stat. Med.\**, vol. 36, no. 5, pp. 770–790, 2017, doi: 10.1002/sim.7171.
- [8] H. Seibold, A. Zeileis, and T. Hothorn, "Subgroup identification from randomized clinical trial data," *\*Stat. Model.\**, vol. 16, no. 4, pp. 289–314, 2016, doi: 10.1177/1471082X16642371.
- [9] A. Gruca and M. Sikora, "RuleMiner: An interactive web tool for rule-based data analysis," *\*J. Biomed. Inform.\**, vol. 123, p. 103948, 2021, doi: 10.1016/j.jbi.2021.103948.
- [10] S. Patel and S. Sahu, "Finding semantic patterns in omics data using concept rule learning," *\*BMC Bioinformatics\**, vol. 22, no. 1, pp. 1–15, 2021, doi: 10.1186/s13040-020-00219-6.
- [11] TCGA Pan-Cancer Clinical Data. "phenotype - Curated clinical data," 2017. [Online]. Available: [https://tcga-pancan-atlas-hub.s3.us-east-1.amazonaws.com/download/Survival\\_SupplementalTable\\_S1\\_20171025\\_xena\\_sp](https://tcga-pancan-atlas-hub.s3.us-east-1.amazonaws.com/download/Survival_SupplementalTable_S1_20171025_xena_sp)